SMART CAMPUS

Plateforme numérique de gestion de la vie universitaire

Dossier technique de conception

Mini-projet : Architecture Web Distribuée, AWD

Validation : Validation intermédiaire, semaine 4

Établissement : ESPRIT

Classe : 4DAS11

Groupe : Groupe 2

Membres de l'équipe

- Chadha Dalhoumi
- Bechir Hbaoubi
- Dhia Trabelsi
- Yassine Gharbi

Année universitaire : 2026-2027

1. Présentation du projet

1.1 Contexte

À l'université, les services de la vie de campus sont gérés séparément. Les demandes de salles et de matériel passent par e-mail ou directement par l'administration. Les événements sont annoncés sur plusieurs groupes différents et les inscriptions ne sont pas centralisées. Les objets perdus sont signalés sur des affiches ou dans des groupes de discussion, et les utilisateurs ne sont pas toujours prévenus quand le statut de leur demande change.

Cette situation provoque des conflits (une salle demandée par deux personnes sur le même créneau), des informations perdues et beaucoup de vérifications manuelles pour l'administration. Ce constat est notre hypothèse de départ. Il pourra être vérifié par de courts entretiens avec des étudiants, des responsables de clubs et des membres de l'administration.

1.2 Problématique

Comment centraliser les principaux services de la vie du campus tout en garantissant la cohérence des données, la sécurité des accès, la traçabilité des actions et l'indépendance des domaines métier ?

Cette question se décompose en plusieurs problèmes :

- éviter les conflits lors de la réservation d'une salle ou d'un équipement ;
- suivre une demande de sa création jusqu'à son approbation ou son refus ;
- gérer les clubs, les événements et les inscriptions sans dépasser la capacité autorisée ;
- diffuser les annonces au bon public et suivre les objets perdus jusqu'à leur restitution ;
- informer les utilisateurs sans créer de dépendance directe entre tous les services ;
- sécuriser les opérations selon le rôle de chacun, sans base de données unique partagée.

1.3 Solution proposée

Smart Campus est une application web unique pour l'utilisateur (interface Angular), mais distribuée sur le plan technique : cinq microservices Spring Boot se partagent les fonctionnalités. Elle couvre cinq domaines.

Domaine	Contenu
Réservations	Bâtiments, salles, matériel, disponibilités et demandes de réservation.
Clubs et événements	Clubs, adhésions, responsables, événements, inscriptions et listes d'attente.
Annonces et articles	Contenus officiels ou associatifs, validation, planification et expiration.
Objets perdus et trouvés	Déclarations, correspondances, réclamations, vérifications et restitutions.
Notifications	Notifications individuelles, rappels et messages en temps réel, créés à partir des événements des autres services.

1.4 Objectifs

Objectifs fonctionnels

- consulter les salles et le matériel disponibles, demander une réservation, l'approuver ou la refuser ;
- gérer les clubs, les adhésions et les événements, avec une capacité limitée et une liste d'attente ;
- publier, planifier et faire expirer des annonces et des articles ;
- déclarer un objet perdu ou trouvé, proposer des correspondances, gérer les réclamations et les restitutions ;
- notifier les utilisateurs lors des changements importants et envoyer des rappels.

Objectifs techniques

- un microservice par capacité métier, avec sa propre base de données ;
- une API Gateway comme point d'entrée, Eureka pour la découverte, Config Server pour les configurations ;
- Keycloak pour l'authentification et les rôles ;
- des appels synchrones (REST, OpenFeign) et asynchrones (RabbitMQ) ;
- le déploiement local avec Docker et Docker Compose.

2. Analyse fonctionnelle

2.1 Acteurs

Acteur	Description	Actions principales
Étudiant	Utilisateur principal	Consulter clubs et événements, s'inscrire, consulter les contenus, déclarer ou réclamer un objet.
Responsable de club	Étudiant avec le rôle métier MANAGER dans un club	Gérer les adhésions, créer des événements et rédiger des contenus pour son club.
Personnel	Enseignant ou membre du personnel	Réserver des ressources, créer un événement institutionnel, publier des annonces officielles.
Administrateur	Valide et supervise	Approuver réservations, clubs et contenus, gérer les ressources, enregistrer les restitutions.
Système de planification	Composant automatique interne	Publier les contenus programmés, faire expirer contenus et objets, envoyer les rappels.
Microservice technique	Service authentifié par un compte de service	Appeler une API interne, publier ou consommer un événement.

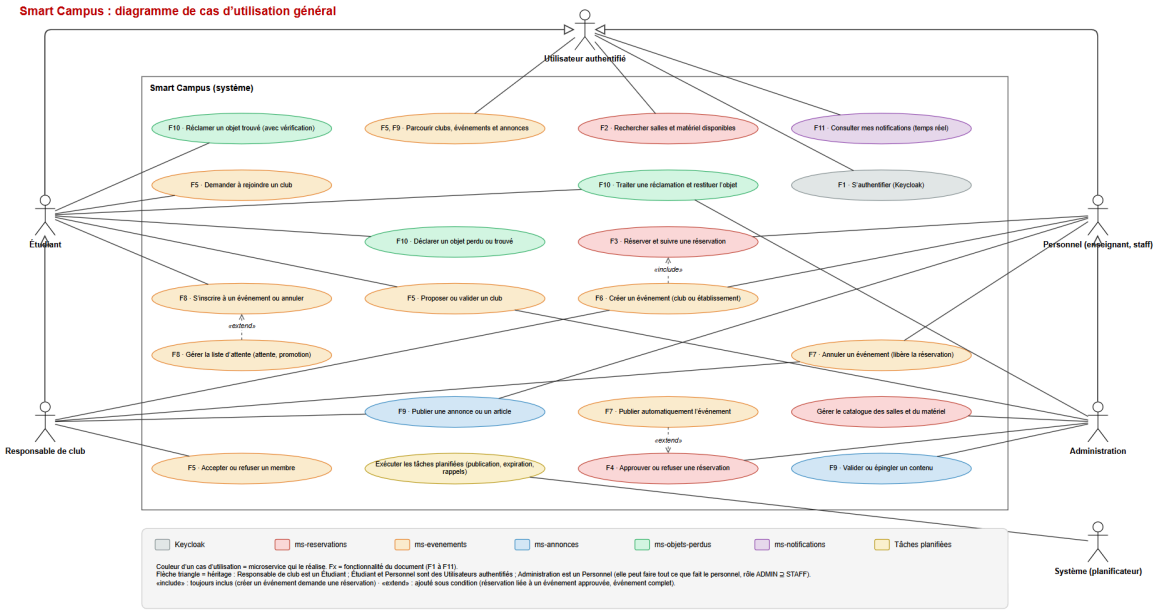


Figure 2.1 : Diagramme de cas d'utilisation général

2.2 Rôles et droits

Keycloak gère trois rôles globaux : **STUDENT**, **STAFF** et **ADMIN**. Ils sont présents dans le jeton JWT.

Le rôle **MANAGER** n'est pas un rôle global. C'est un rôle métier géré par ms-evenements, car il dépend du club : un même utilisateur peut être responsable d'un club, simple membre d'un deuxième et absent d'un troisième. Il est donc stocké dans l'adhésion et vérifié pour le club concerné.

Opération	STUDENT	STAFF	ADMIN	MANAGER du club
Consulter clubs, événements et contenus, s'inscrire, déclarer un objet, réclamer, consulter ses notifications	Oui	Oui	Oui	Oui
Approuver un club	Non	Non	Oui	Non
Accepter une adhésion	Non	Non	Oui	Oui, son club
Créer un événement de club	Non	Non	Oui	Oui, son club
Créer un événement institutionnel	Non	Oui	Oui	Non
Créer une réservation manuelle	Non	Oui	Oui	Non
Approuver une réservation	Non	Non	Oui	Non
Créer un contenu officiel	Non	Oui	Oui	Non
Créer un contenu de club	Non	Non	Oui	Oui, son club
Approuver un contenu de club	Non	Non	Oui	Non
Enregistrer une restitution	Non	Non	Oui	Non

2.3 Fonctionnalités principales

Réf.	Fonctionnalité	Responsable
F1	S'authentifier et obtenir les rôles globaux	Keycloak
F2	Rechercher les salles et le matériel disponibles sur un créneau	ms-reservations
F3	Réserver une salle et du matériel dans une même demande	ms-reservations
F4	Approuver, refuser ou annuler une réservation	ms-reservations
F5	Consulter les clubs et gérer les demandes d'adhésion	ms-evenements
F6	Créer un événement de club ou institutionnel	ms-evenements
F7	Publier ou annuler un événement selon la décision de réservation	ms-evenements, ms-reservations
F8	Gérer inscriptions, capacité et liste d'attente	ms-evenements
F9	Créer, valider, programmer et publier des annonces ou articles	ms-annonces
F10	Déclarer un objet, trouver des correspondances, réclamer et restituer	ms-objets-perdus
F11	Créer, conserver et envoyer les notifications et les rappels	ms-notifications

Une fonctionnalité peut faire intervenir plusieurs services, mais un seul service reste responsable de la décision métier principale.

2.4 Cas d'utilisation prioritaires

UC-01 : créer un événement avec réservation

Acteurs	Principal : responsable de club ou membre du personnel. Secondaires : administration, ms-reservations, ms-notifications.
Préconditions	Utilisateur authentifié. Club actif (événement de club). Rôle MANAGER du club, ou STAFF/ADMIN pour un événement institutionnel. Date de fin postérieure à la date de début ; capacité positive.
Scénario nominal	1. L'utilisateur saisit l'événement, choisit une salle et le matériel. 2. ms-evenements vérifie ses droits et la capacité de la salle (appel à ms-reservations). 3. ms-evenements demande la création d'une réservation à ms-reservations. 4. ms-reservations vérifie la salle et le matériel, puis crée une réservation PENDING. 5. ms-evenements crée l'événement au statut PENDING_RESERVATION. 6. L'administration approuve ; ms-reservations publie ReservationApproved. 7. ms-evenements passe l'événement à PUBLISHED et publie EventPublished. 8. ms-notifications crée les notifications des membres concernés.
Alternatives	Salle ou matériel indisponible : 409 Conflict, rien n'est conservé. Rôle non valide : 403 Forbidden. ms-reservations indisponible : 503, l'événement reste en brouillon. Réservation refusée : l'événement passe à RESERVATION_REFUSED.
Postcondition	L'événement est publié uniquement après approbation de la réservation (sauf événement institutionnel approuvé automatiquement par un administrateur).

UC-02 : s'inscrire à un événement

Acteur	Utilisateur authentifié.
Préconditions	Événement publié, inscriptions ouvertes, utilisateur ni inscrit ni en liste d'attente.
Scénario nominal	1. L'utilisateur demande son inscription. 2. ms-evenements verrouille la ligne de l'événement le temps de la transaction et compte les places occupées. 3. Une place est libre : l'inscription est créée au statut REGISTERED. 4. RegistrationConfirmed est publié ; ms-notifications crée une confirmation et programme un rappel.
Variantes	Capacité atteinte : inscription WAITLISTED avec un rang dans la liste d'attente. Annulation : l'inscription passe à CANCELLED, le premier de la liste d'attente devient REGISTERED, WaitlistPromoted est publié et la personne est notifiée.
Postcondition	L'événement ne dépasse jamais sa capacité maximale.

UC-03 : publier un contenu de club

Acteurs	Principal : responsable de club. Secondaire : administrateur.
Préconditions	Utilisateur authentifié, club actif, utilisateur MANAGER de ce club.
Scénario nominal	1. L'utilisateur saisit le contenu. 2. ms-annonces demande à ms-evenements de vérifier le rôle MANAGER. 3. Le contenu est créé au statut PENDING_REVIEW. 4. L'administration l'approuve : il passe à PUBLISHED, immédiatement ou à la date publishAt. 5. ContentApproved est publié et l'auteur est notifié.
Alternatives	Utilisateur non responsable du club : 403 Forbidden. ms-evenements indisponible : 503 Service Unavailable. Contenu refusé : statut REJECTED et événement ContentRejected.

UC-04 : réclamer un objet trouvé

Acteurs	Principal : utilisateur qui se dit propriétaire. Secondaires : déclarant de l'objet trouvé, administration.
Préconditions	L'objet trouvé existe, son statut permet une réclamation, il n'a pas déjà été restitué, l'utilisateur est authentifié.
Scénario nominal	1. L'utilisateur consulte une correspondance proposée et répond à la question de vérification. 2. Une réclamation PENDING est ajoutée au document de l'objet trouvé. 3. Le déclarant ou l'administration l'examine et l'approuve ; ClaimApproved est publié. 4. L'objet est remis au bureau de l'administration, qui enregistre la restitution. 5. L'objet passe à RETURNED et ItemReturned est publié.
Alternatives	Réponse insuffisante : réclamation REJECTED. Objet déjà restitué : 409 Conflict. Utilisateur non autorisé à consulter les correspondances : 403 Forbidden.

2.5 Règles fonctionnelles transversales

Règle	Description
Authentification	Toute opération de modification exige un JWT valide délivré par Keycloak.
Identité	L'identifiant de l'utilisateur est lu dans le claim sub du jeton, jamais dans le corps de la requête.
Propriété des données	Chaque microservice est seul propriétaire de ses données et de ses statuts. Aucun service ne lit ni ne modifie la base d'un autre.
Références externes	Un identifiant appartenant à un autre service est enregistré comme simple référence, sans clé étrangère entre bases.
Idempotence	Idempotency-Key pour la création d'une réservation demandée par un autre service ; messageId pour les événements RabbitMQ.
Notifications non bloquantes	Si ms-notifications est indisponible, l'opération du service producteur n'est pas interrompue.
Traçabilit	Les décisions sensibles (approbation d'une réservation, restitution d'un objet) conservent l'utilisateur et la date.

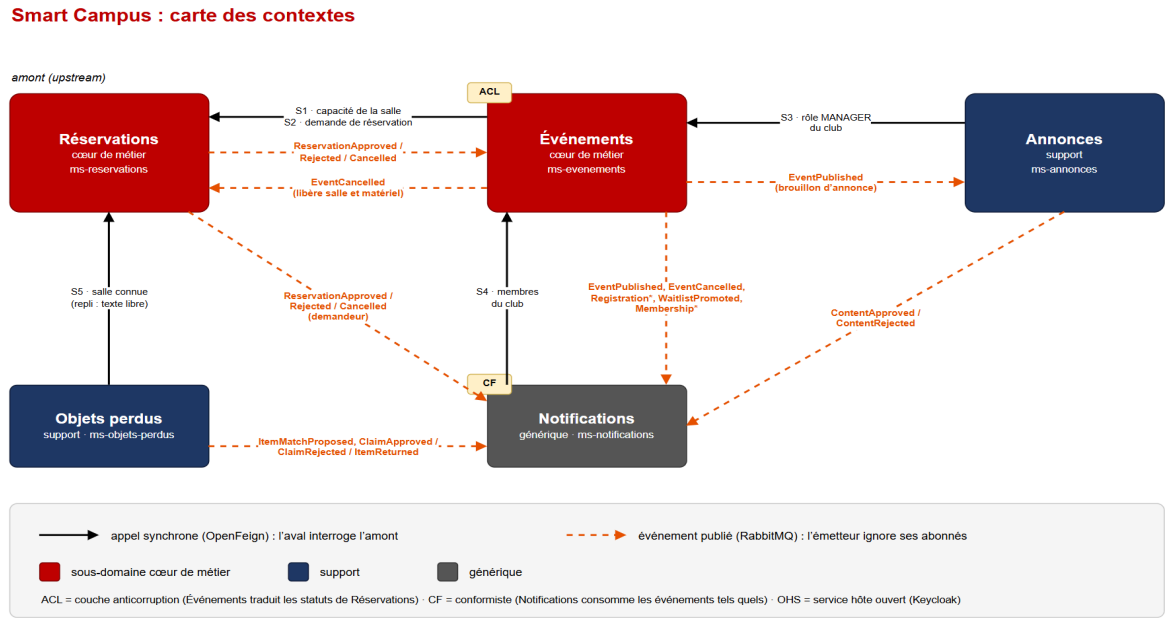
3. Découpage en microservices

3.1 Principe de découpage

Le découpage repose sur les capacités métier (contextes bornés) et non sur les tables ou les écrans. Chaque microservice a une responsabilité principale, possède ses données, contrôle ses statuts, expose un contrat d'API et ne partage pas sa base avec un autre service.

Nous retenons quatre microservices métier, un par membre, et un cinquième microservice commun (ms-notifications) développé par toute l'équipe. API Gateway, Eureka, Config Server, Keycloak et RabbitMQ sont des composants d'infrastructure, pas des microservices métier.

Microservice	Capacité métier	Base de données
ms-reservations	Salles, matériel et réservations	PostgreSQL
ms-evenements	Clubs, adhésions, événements et inscriptions	MySQL
ms-annonces	Annonces et articles de blog	H2 (mode fichier)
ms-objets-perdus	Objets perdus et trouvés, réclamations, restitutions	MongoDB
ms-notifications	Notifications individuelles, préférences et rappels	H2 (mode fichier)



3.3 Responsabilités et choix de la base par service

Chaque microservice gère une capacité métier précise et possède sa propre base, choisie selon la nature de ses données.

- **ms-reservations (PostgreSQL)**

    - Gère les bâtiments, les salles, le matériel et les réservations (création, approbation, refus, annulation). Salles et matériel sont regroupés pour réserver « tout ou rien » dans une seule transaction locale.
    - PostgreSQL apporte les transactions, les verrous de lignes pour le matériel et une contrainte d'exclusion (tstzrange) qui interdit deux réservations qui se chevauchent sur la même salle.
- **ms-evenements (MySQL)**
    - Gère les clubs, les adhésions, le rôle MANAGER, les événements, les inscriptions et les listes d'attente. Ils sont regroupés pour que la vérification du rôle MANAGER reste locale.
    - MySQL convient à ces données très relationnelles. InnoDB (transactions, verrous) évite de dépasser la capacité d'un événement, et les contraintes d'unicité évitent les doubles inscriptions.
- **ms-annonces (H2 en mode fichier)**
    - Gère les annonces et les articles : brouillons, validation, publication programmée, expiration et archivage. Une annonce vise une audience, alors qu'une notification est un message individuel, d'où deux services distincts.
    - H2 suffit pour un modèle simple, un volume modeste et des écritures peu sensibles. C'est un choix de prototype (pas de réplication), avec une migration possible vers PostgreSQL ou MySQL.
- **ms-objets-perdus (MongoDB)**
    - Gère les déclarations LOST et FOUND, les correspondances, les réclamations et les restitutions.
    - MongoDB permet des caractéristiques variables selon l'objet (champ attributes), intègre réclamations et restitution dans le même document et offre un index textuel. Sans transaction multidocument, les mises à jour sont conditionnelles et idempotentes.
- **ms-notifications (H2 en mode fichier, microservice commun)**
    - Consomme les événements, crée les notifications, gère préférences et rappels, et livre en temps réel par SSE. Il met en œuvre RabbitMQ (files durables, acquittement manuel), l'idempotence, les nouvelles tentatives et la Dead Letter Queue.
    - H2 est choisi pour le prototype (schéma limité, volume modeste), avec la même limite que ms-annonces.

3.4 Isolation des données

- Aucune base n'est partagée et aucun service n'interroge la base d'un autre.
- Les identifiants externes sont de simples références, sans clé étrangère entre bases.
- Les échanges passent uniquement par REST ou RabbitMQ, et les deux services sur H2 ont chacun leur fichier et leur volume Docker.

4. Architecture technique globale

4.1 Vue d'ensemble

L'utilisateur accède à une seule application Angular. Toutes les requêtes passent par l'API Gateway, qui les dirige vers le bon microservice grâce à Eureka. Chaque microservice travaille sur sa propre base et publie des événements RabbitMQ pour les traitements secondaires.

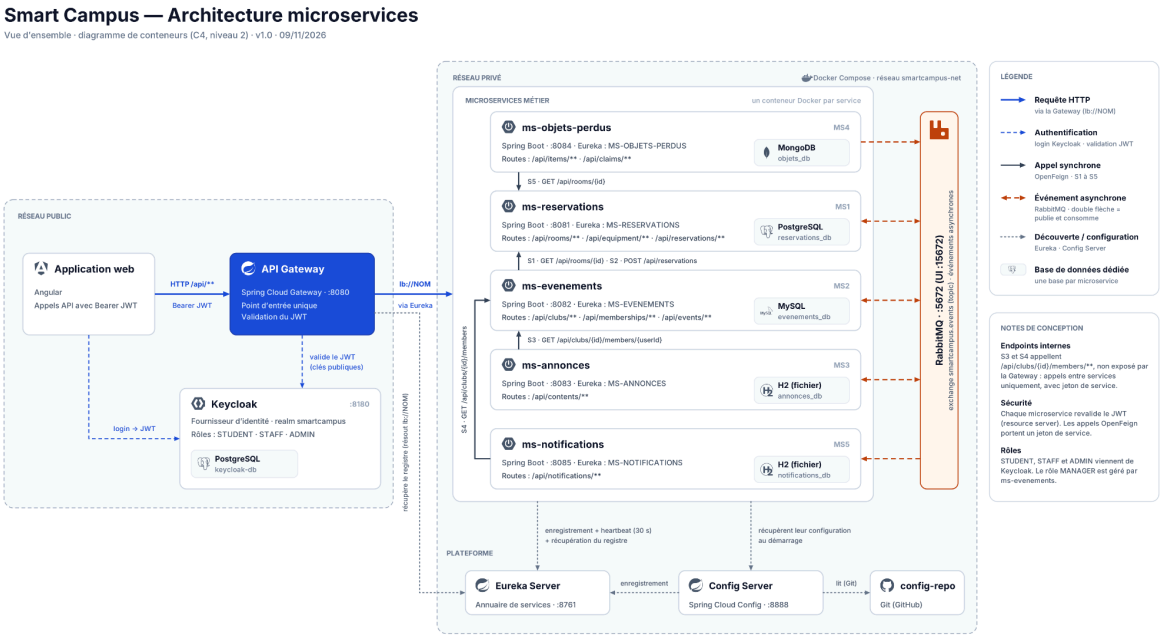


Figure 4.1 : Diagramme d'architecture globale de Smart Campus

4.2 Composants

Composant	Rôle
Frontend Angular	Interface unique organisée en modules (authentification, réservations, clubs et événements, annonces, objets perdus, notifications, administration). Elle s'intègre à Keycloak, joint le JWT aux requêtes et consomme le flux SSE. Elle ne contient pas de décision métier sensible : les règles sont toujours revérifiées par les microservices.
API Gateway	Point d'entrée unique : routage des requêtes, transmission du JWT, règles de sécurité générales, en-têtes communs et masquage des adresses internes des microservices.
Eureka Server	Registre de découverte : chaque service s'enregistre avec son nom, son adresse, son port et son état. Il évite les adresses codées en dur et facilite les appels OpenFeign.
Config Server	Centralise les configurations (Eureka, RabbitMQ, journalisation, délais d'attente, noms des files, résilience). Les secrets ne sont jamais dans le dépôt : variables d'environnement ou fichiers locaux exclus par .gitignore.
Keycloak	Authentification et rôles globaux STUDENT, STAFF et ADMIN. Il délivre le jeton JWT.
RabbitMQ	Broker des communications asynchrones : il découple producteurs et consommateurs et gère les nouvelles tentatives et les messages en échec.

Composant	Rôle
Docker Compose	Reproduit l'environnement local : PostgreSQL, MySQL, MongoDB, RabbitMQ, Keycloak, Eureka, Config Server, API Gateway, les cinq microservices et éventuellement Angular.

4.3 Routage de la Gateway

Les routes utilisent les noms logiques d'Eureka (par exemple lb://MS-RESERVATIONS), ce qui évite toute adresse codée en dur.

Route	Microservice
`/api/reservations/**, /api/buildings/**, /api/rooms/**`	ms-reservations
`/api/clubs/**, /api/events/**`	ms-evenements
`/api/contents/**`	ms-annonces
`/api/items/**`	ms-objets-perdus
`/api/notifications/**, /api/notification-preferences/**`	ms-notifications

4.4 Parcours d'une requête

1. L'utilisateur s'authentifie auprès de Keycloak, qui délivre un jeton JWT.
2. Angular joint le jeton à chaque requête.
3. L'API Gateway identifie le microservice destinataire et la localise grâce à Eureka.
4. Le microservice valide le jeton, vérifie les autorisations et traite la requête dans sa propre base.
5. Si la décision métier doit déclencher un traitement secondaire, le service publie un événement RabbitMQ.
6. Les consommateurs traitent l'événement de façon asynchrone ; ms-notifications peut alors envoyer une notification en temps réel par SSE.

4.5 Technologies choisies

Domaine	Technologie
Frontend	Angular
Backend	Spring Boot (cinq microservices)
Entrée, découverte, configuration	API Gateway, Eureka Server, Config Server
Sécurité	Keycloak (OAuth 2.0, JWT, Authorization Code avec PKCE)
Communication synchrone	REST et OpenFeign
Communication asynchrone	RabbitMQ
Temps réel	Server-Sent Events (SSE)
Résilience	Resilience4j (circuit breaker), délais d'attente, Dead Letter Queue
Bases de données	PostgreSQL, MySQL, MongoDB, H2

Domaine	Technologie
Conteneurisation	Docker et Docker Compose
Gestion de projet	Git, GitHub et GitHub Projects

4.6 Sécurité

- **Frontend :** Angular utilise le flux Authorization Code avec PKCE. Il ne contient jamais de secret de client confidentiel.
- **Validation du JWT :** chaque microservice est configuré comme Resource Server (signature, expiration, émetteur, rôles). La validation par la seule Gateway ne suffit pas.
- **Deux niveaux d'autorisation :** les rôles globaux (Keycloak) et les règles métier locales, par exemple « seul le responsable du club peut modifier un événement de ce club ».
- **Appels entre services :** comptes de service (clients confidentiels) et routes internes non exposées par la Gateway.
- **Secrets :** jamais dans le dépôt public.

5. Communications entre les microservices

5.1 Choix du mode de communication

Une communication est **synchrone** (REST et OpenFeign) quand l'appelant a besoin d'une réponse immédiate pour continuer. Elle est **asynchrone** (RabbitMQ) quand le traitement peut se faire après la décision métier principale. Un événement RabbitMQ annonce un fait déjà enregistré : le consommateur ne peut pas modifier après coup la transaction du producteur.

5.2 Appels synchrones

Appelant	Appelé	Opération	Justification	Si l'appelé est indisponible
ms-evenements	ms-reservations	Consulter une salle	Vérifier la capacité immédiatement	503, l'événement reste en brouillon
ms-evenements	ms-reservations	Vérifier une disponibilité	Réponse nécessaire avant la demande	503, l'événement reste en brouillon
ms-evenements	ms-reservations	Créer une réservation	L'identifiant est nécessaire à l'événement (Idempotency-Key contre les doublons)	503, l'événement reste en brouillon
ms-annonces	ms-evenements	Vérifier le rôle MANAGER	Autorisation requise avant la création	503 : un droit non vérifié n'est jamais supposé valide
ms-notifications	ms-evenements	Récupérer les membres actifs d'un club	Déterminer les destinataires	Message réessayé, puis envoyé en Dead Letter Queue

5.3 Événements asynchrones

Événement	Producteur	Consommateurs	Effet
ReservationApproved	ms-réservations	ms-evenements, ms-notifications	Publier l'événement et informer
ReservationRejected	ms-reservations	ms-evenements, ms-notifications	Refuser la publication et informer
ReservationCancelled	ms-reservations	ms-evenements, ms-notifications	Mettre à jour l'événement et informer
ReservationExpired	ms-reservations	ms-evenements	Passer l'événement à RESERVATION_REFUSED
EventPublished	ms-evenements	ms-annonces, ms-notifications	Créer un brouillon de contenu et informer
EventCancelled	ms-evenements	ms-reservations, ms-notifications	Libérer les ressources et informer

Événement	Producteur	Consommateurs	Effet
RegistrationConfirmed	ms-evenements	ms-notifications	Confirmer l'inscription, programmer un rappel
RegistrationCancelled	ms-evenements	ms-notifications	Annuler le rappel éventuel
WaitlistPromoted	ms-evenements	ms-notifications	Informer la personne promue
ContentApproved	ms-annonces	ms-notifications	Informer l'auteur
ContentRejected	ms-annonces	ms-notifications	Informer l'auteur
ItemMatchProposed	ms-objets-perdus	ms-notifications	Informer le propriétaire potentiel
ClaimApproved	ms-objets-perdus	ms-notifications	Informer de la validation
ClaimRejected	ms-objets-perdus	ms-notifications	Informer du refus
ItemReturned	ms-objets-perdus	ms-notifications	Confirmer la restitution

5.4 Gestion des pannes et fiabilité

- délai d'attente (timeout) sur tous les appels synchrones, et circuit breaker Resilience4j quand c'est nécessaire ;
- nouvelles tentatives limitées sur les événements, puis Dead Letter Queue pour les messages en échec ;
- files RabbitMQ durables : les messages sont conservés tant qu'ils ne sont pas traités ;
- traitement idempotent : Idempotency-Key pour les réservations, messageId pour les messages ;
- une panne de ms-notifications ne bloque jamais l'opération du service producteur.

6. Conception détaillée des microservices

Tous les endpoints exigent un jeton JWT valide (401 sinon) ; ce code n'est donc pas répété dans les tableaux. Les routes marquées « interne » ne sont pas exposées par la Gateway.

6.1 ms-reservations

Le service garantit qu'une salle n'est pas utilisée par deux réservations actives sur le même créneau et que la quantité de matériel réservée ne dépasse pas le stock.

6.1.1 Modèle de données

Entité	Attributs principaux
Building	id, name, code (unique), address, active
Room	id, buildingId, name, capacity, type, active
EquipmentType	id, name, totalQuantity, active
Reservation	id, requesterId, roomId, sourceType, sourceRef, startAt, endAt, status, expiresAt, createdAt, updatedAt
ReservationLine	id, reservationId, equipmentTypeId, quantity
ReservationHistory	Historique des changements de statut (statut, utilisateur, date)
ProcessedMessage	messageId : évite de traiter deux fois un même message

Statuts d'une réservation : PENDING, APPROVED, REJECTED, EXPIRED, CANCELLED.

Règles métier

- la date de fin est postérieure à la date de début ; une salle inactive ne peut pas être réservée ;
- la capacité demandée ne dépasse pas celle de la salle ;
- PENDING et APPROVED bloquent le créneau ; une réservation PENDING peut expirer sans décision ;
- le matériel est validé dans la même transaction que la salle (tout ou rien) ;
- une réservation ne s'approuve qu'à partir de PENDING ; l'annulation libère la salle et le matériel.

Chevauchements : une contrainte d'exclusion PostgreSQL sur un intervalle tstzrange refuse l'insertion si, pour la même salle, deux intervalles se chevauchent avec un statut bloquant. L'intervalle inclut le début et exclut la fin : une réservation peut donc commencer quand la précédente se termine.

Matériel : pour chaque type, la ligne EquipmentType est verrouillée, la quantité déjà réservée sur le créneau est calculée puis comparée à totalQuantity ; en cas de manque, la transaction est annulée (rollback).

6.1.2 API REST

Méthode	URI	Fonction	Codes
GET	`/api/buildings`	Lister les bâtiments	200

Méthode	URI	Fonction	Codes
GET	`/api/rooms`	Rechercher des salles	200, 400
GET	`/api/rooms/{id}`	Consulter une salle	200, 404
GET	`/api/availability`	Vérifier les ressources sur un créneau	200, 400
POST	`/api/reservations`	Créer une réservation (en-tête Idempotency-Key)	201, 400, 409
GET	`/api/reservations/{id}`	Consulter une réservation	200, 403, 404
PATCH	`/api/reservations/{id}/approve`	Approuver	200, 403, 404, 409
PATCH	`/api/reservations/{id}/reject`	Refuser	200, 403, 404, 409
PATCH	`/api/reservations/{id}/cancel`	Annuler	200, 403, 404, 409

6.1.3 Événements et interactions

- **Publie :** ReservationApproved, ReservationRejected, ReservationCancelled, ReservationExpired.
- **Consomme :** EventCancelled (annule la réservation associée et libère les ressources).
- **Appelé par :** ms-evenements (consulter une salle, vérifier la disponibilité, créer une réservation). Ce service n'appelle aucun autre service en synchrone.

6.1.4 Diagrammes

ms-reservations : diagramme de classes

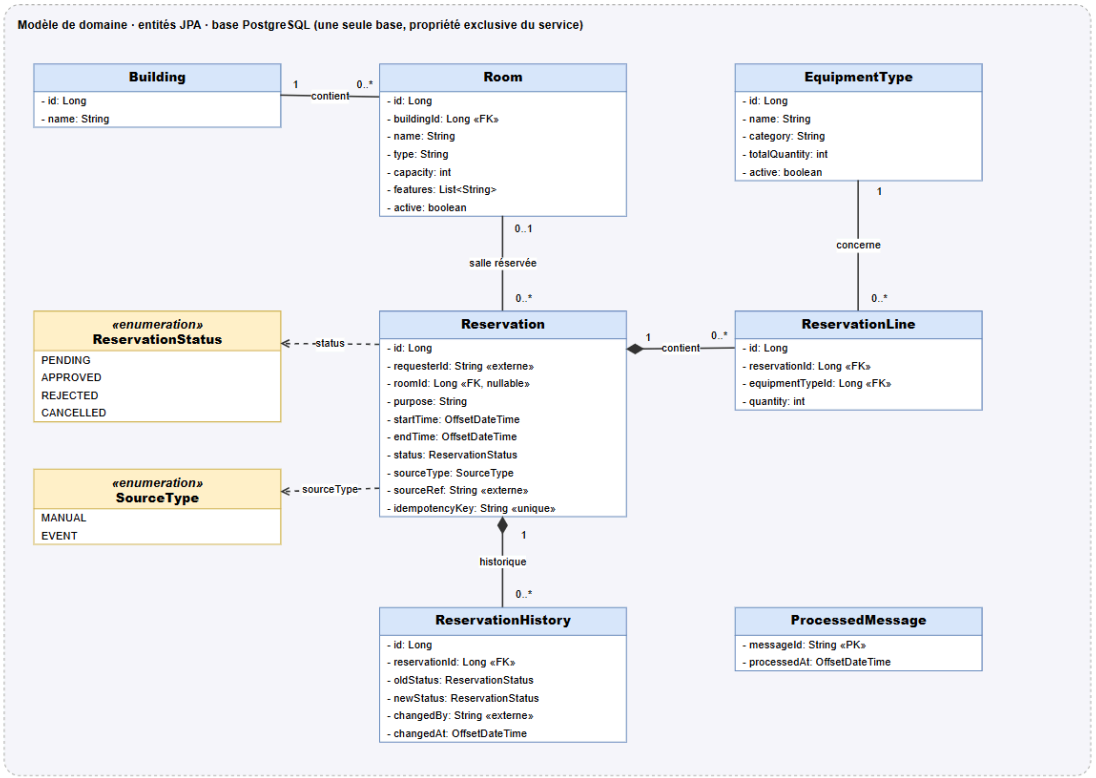


Figure 6.1 : Diagramme de classes de ms-reservations

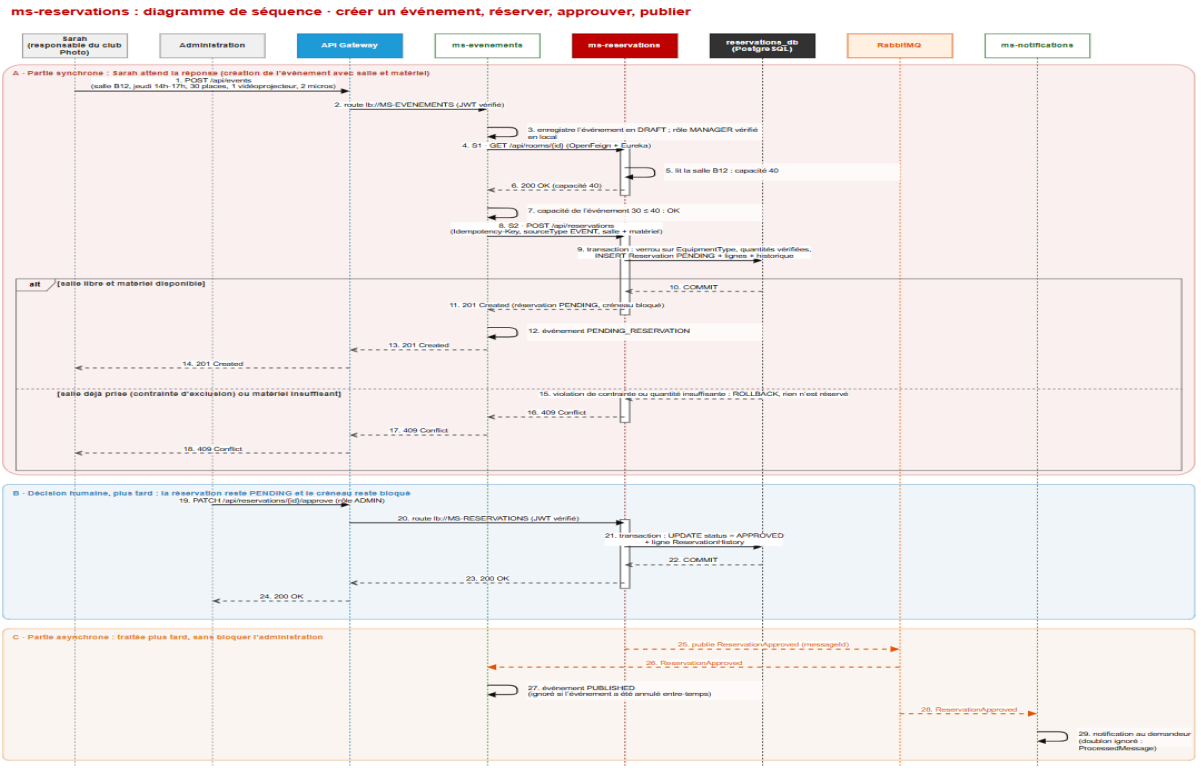


Figure 6.2 : Diagramme de séquence de ms-reservations

6.2 ms-evenements

Le service gère les clubs, les adhésions, les événements, les inscriptions et les listes d'attente. Il est la source de vérité du rôle MANAGER.

6.2.1 Modèle de données

Entité	Attributs principaux
Club	id, name, description, category, status, createdBy, createdAt
Membership	id, clubId, userId, role, status, joinedAt
Event	id, clubId (facultatif), organizerId, title, description, startAt, endAt, capacity, roomId, reservationId, status, createdAt
Registration	id, eventId, userId, status, waitlistPosition, registeredAt

Énumération	Valeurs
ClubStatus	PENDING, ACTIVE, REJECTED, SUSPENDED
MembershipRole	MEMBER, MANAGER
MembershipStatus	PENDING, ACTIVE, REJECTED, LEFT
EventStatus	DRAFT, PENDING_RESERVATION, PUBLISHED, RESERVATION_REFUSED, CANCELLED, COMPLETED

Énumération	Valeurs
RegistrationStatus	REGISTERED, WAITLISTED, CANCELLED

6.2.2 API REST

Méthode	URI	Fonction	Codes
POST	`/api/clubs`	Proposer un club	201, 400
GET	`/api/clubs`	Lister les clubs	200
GET	`/api/clubs/{id}`	Consulter un club	200, 404
PATCH	`/api/clubs/{id}/approve`	Approuver un club	200, 403, 404
POST	`/api/clubs/{id}/memberships`	Demander une adhésion	201, 409
PATCH	`/api/clubs/{id}/memberships/{membershipId}/approve`	Accepter une adhésion	200, 403, 404
POST	`/api/events`	Créer un événement	201, 400, 403, 409, 503
GET	`/api/events`	Rechercher des événements	200
GET	`/api/events/{id}`	Consulter un événement	200, 404
PATCH	`/api/events/{id}`	Modifier un événement	200, 403, 404, 409
POST	`/api/events/{id}/registrations`	S'inscrire	201, 409
DELETE	`/api/events/{id}/registrations/me`	Annuler son inscription	204, 404
PATCH	`/api/events/{id}/cancel`	Annuler l'événement	200, 403, 404, 409
GET	`/internal/clubs/{id}/members/{userId}/role`	Interne : rôle d'un utilisateur dans un club	200, 404
GET	`/internal/clubs/{id}/members`	Interne : membres actifs d'un club	200, 404

6.2.3 Événements et interactions

- **Publie :** EventPublished, EventCancelled, RegistrationConfirmed, RegistrationCancelled, WaitlistPromoted.
- **Consomme :** ReservationApproved, ReservationRejected, ReservationCancelled, ReservationExpired.
- **Appelle (synchrone) :** ms-reservations pour consulter une salle, vérifier la disponibilité et créer une réservation.
- **Appelé par :** ms-annonces (rôle MANAGER) et ms-notifications (membres actifs d'un club), via les routes internes.

6.2.4 Diagrammes

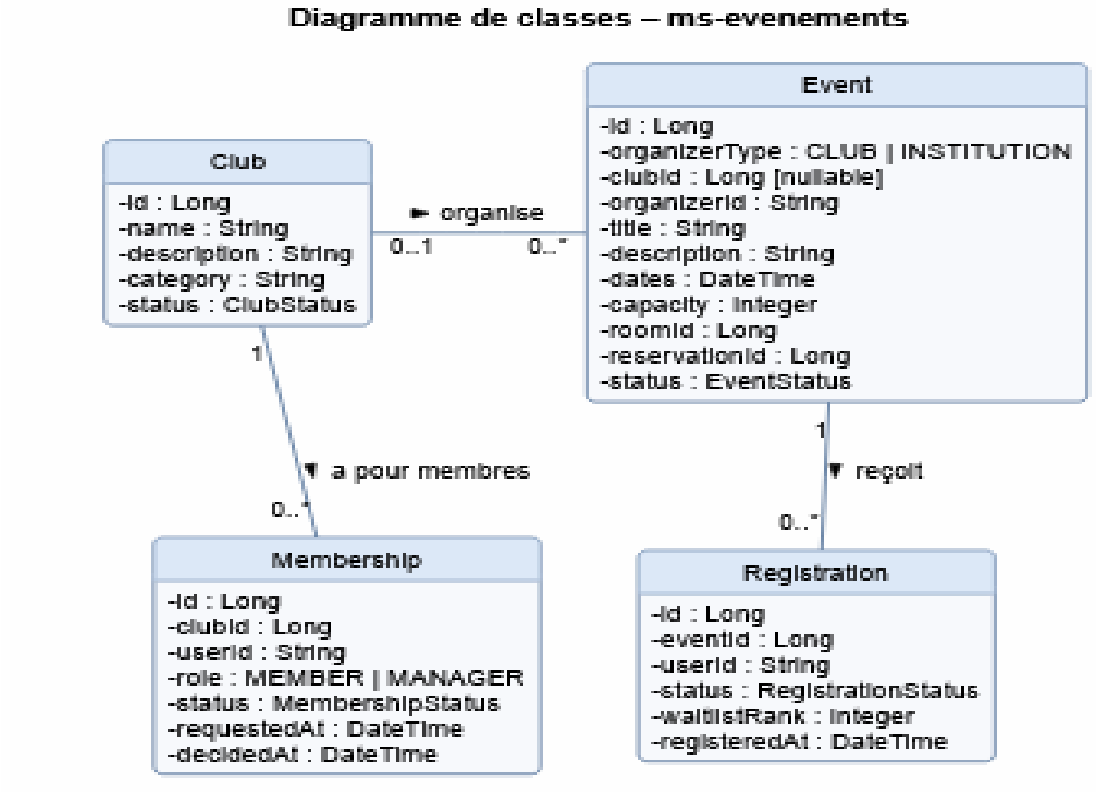


Figure 6.3 : Diagramme de classes de ms-evenements

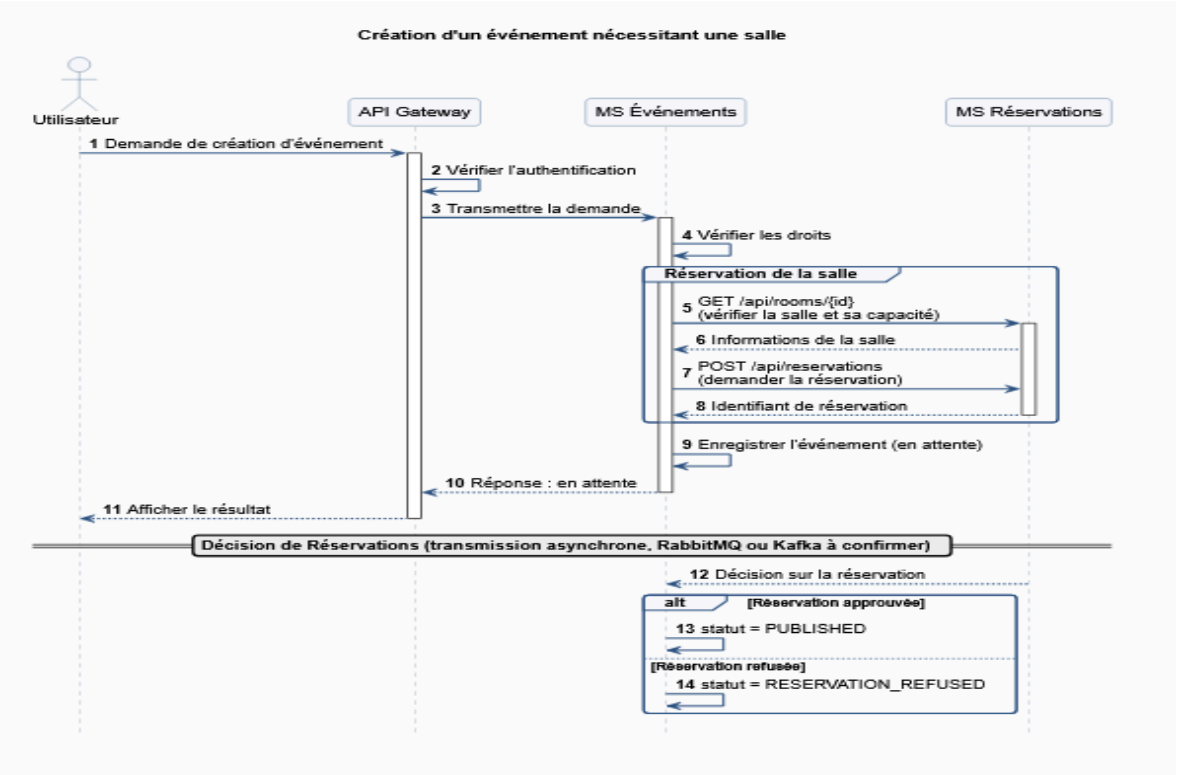


Figure 6.4 : Diagramme de séquence de ms-evenements

6.3 ms-annonces

Le service gère les annonces et les articles diffusés sur le campus, de la rédaction à l'archivage.

6.3.1 Modèle de données

Entité	Attributs principaux
Content	id, kind, title, body, authorId, clubId (facultatif), audience, status, pinned, publishAt, expiresAt, createdAt, updatedAt
ContentTag	id, contentId, tag

Un même Content représente une annonce ou un article de blog, selon l'attribut kind.

Énumération	Valeurs
ContentKind	ANNOUNCEMENT, BLOG_ARTICLE
AudienceType	ALL, STUDENTS, STAFF, CLUB_MEMBERS
ContentStatus	DRAFT, PENDING_REVIEW, SCHEDULED, PUBLISHED, REJECTED, EXPIRED, ARCHIVED

Règles métier

- le titre et le contenu sont obligatoires ; un contenu de club exige un clubId et un auteur MANAGER de ce club ;
- un contenu officiel peut être créé par STAFF ou ADMIN ;
- un contenu de club doit être approuvé avant publication ;
- publishAt doit être dans le futur pour le statut SCHEDULED, et expiresAt postérieure à publishAt.

6.3.2 API REST

Méthode	URI	Fonction	Codes
POST	`/api/contents`	Créer un contenu	201, 400, 403, 503
GET	`/api/contents`	Rechercher des contenus	200
GET	`/api/contents/{id}`	Consulter un contenu	200, 404
PATCH	`/api/contents/{id}`	Modifier un contenu	200, 403, 404, 409
PATCH	`/api/contents/{id}/submit`	Soumettre à validation	200, 404, 409
PATCH	`/api/contents/{id}/approve`	Approuver	200, 403, 404, 409
PATCH	`/api/contents/{id}/reject`	Refuser	200, 403, 404, 409
PATCH	`/api/contents/{id}/archive`	Archiver	200, 403, 404

6.3.3 Événements et interactions

- **Publie :** ContentApproved, ContentRejected.
- **Consomme :** EventPublished (crée un contenu en brouillon lié à l'événement, pour éviter une publication automatique incorrecte).
- **Appelle (synchrone) :** ms-evenements pour vérifier le rôle MANAGER. S'il est indisponible, la création d'un contenu de club retourne 503.

6.3.4 Diagrammes

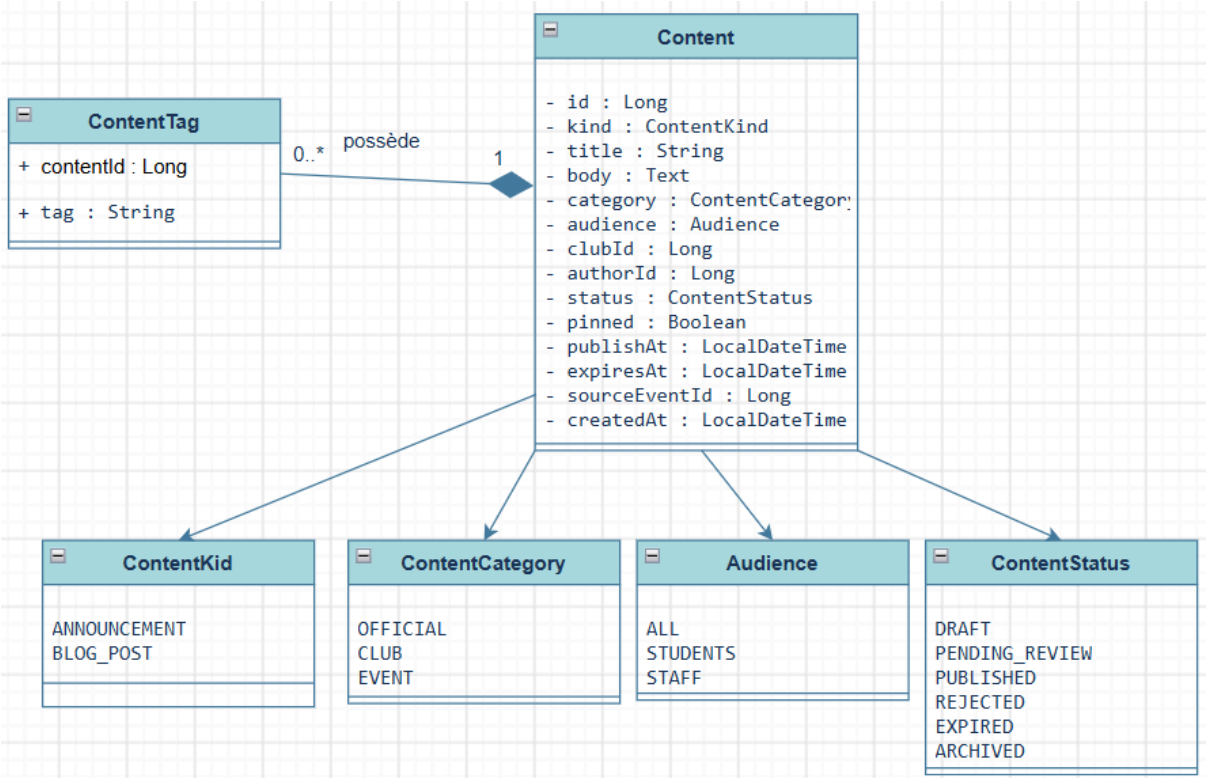


Figure 6.5 : Diagramme de classes de ms-annonces

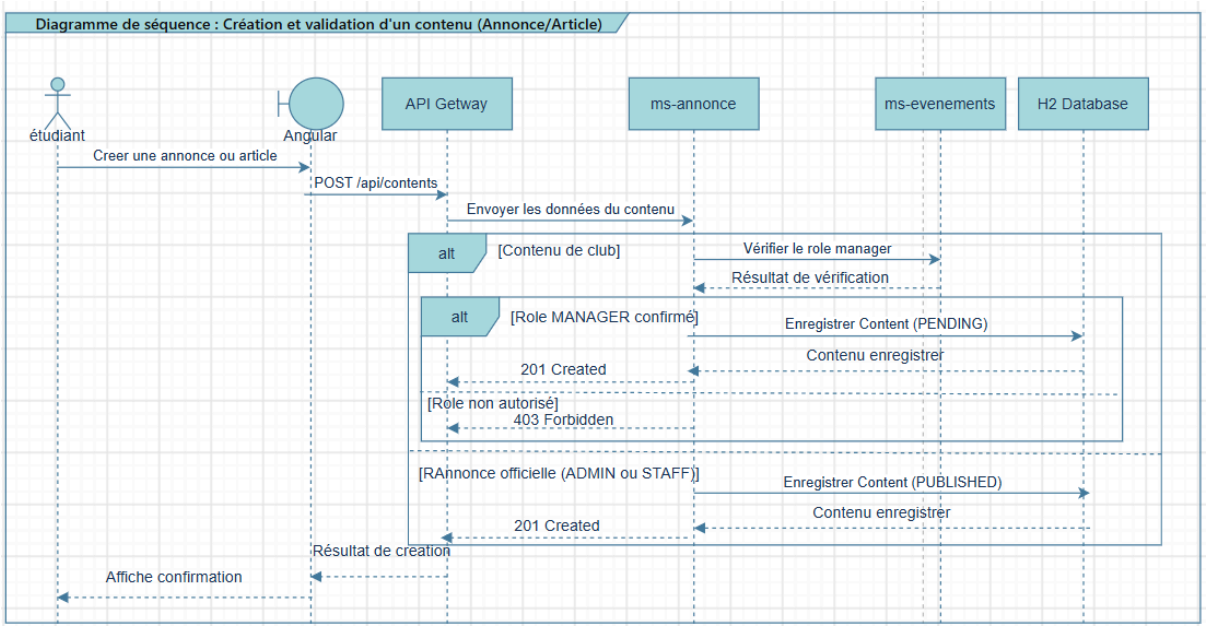


Figure 6.6 : Diagramme de séquence de ms-annonces

6.4 ms-objets-perdus

Le service gère les déclarations d'objets perdus et trouvés, les propositions de correspondance, les réclamations et les restitutions.

6.4.1 Modèle de données

Le document principal est **Item**. Les réclamations (Claim) et la restitution (Handover) sont intégrées dans ce document.

Élément	Attributs principaux
Item	id, type (LOST ou FOUND), ownerUserId, category, title, description, attributes (champs variables), occurredAt, locationText, roomId, photoReference, verificationQuestion, verificationAnswerHash, status, matches, claims, handover, createdAt, updatedAt, expiresAt
Claim	id, claimantId, answer, status (PENDING, APPROVED, REJECTED), createdAt, processedAt, processedBy
Handover	handedOverAt, recordedBy, claimantId, notes

Statuts d'un objet : OPEN, MATCH_PROPOSED, CLAIM_PENDING, CLAIM_APPROVED, RETURNED, EXPIRED, CLOSED.

Calcul de la correspondance

Un score sur 100 est calculé : même catégorie (40), dates proches (20), lieu compatible (15), mots communs (15), attributs communs (10). Une correspondance est proposée à partir de 60/100. Le score propose un rapprochement mais ne prouve pas la propriété de l'objet.

Cohérence sans transaction multidocument : la proposition est conservée dans le document LOST avec le foundItemId, l'état de l'objet FOUND est vérifié avant une réclamation et les mises à jour sont conditionnelles et idempotentes. On évite de dupliquer la même correspondance dans les deux documents.

6.4.2 API REST

Méthode	URI	Fonction	Codes
POST	`/api/items`	Déclarer un objet	201, 400
GET	`/api/items`	Rechercher des objets	200
GET	`/api/items/{id}`	Consulter un objet	200, 404
PATCH	`/api/items/{id}`	Modifier un objet	200, 403, 404, 409
POST	`/api/items/{id}/matches`	Calculer ou proposer des correspondances	201, 403
POST	`/api/items/{id}/claims`	Déposer une réclamation	201, 409
PATCH	`/api/items/{id}/claims/{claimId}/approve`	Approuver une réclamation	200, 403, 409
PATCH	`/api/items/{id}/claims/{claimId}/reject`	Refuser une réclamation	200, 403, 409
POST	`/api/items/{id}/return`	Enregistrer la restitution	200, 403, 409

6.4.3 Événements et interactions

- **Publie :** ItemMatchProposed, ClaimApproved, ClaimRejected, ItemReturned.
- **Consomme :** aucun événement.
- **Appels synchrones :** aucun. Le service reste autonome ; seule ms-notifications réagit à ses événements.

6.4.4 Diagrammes

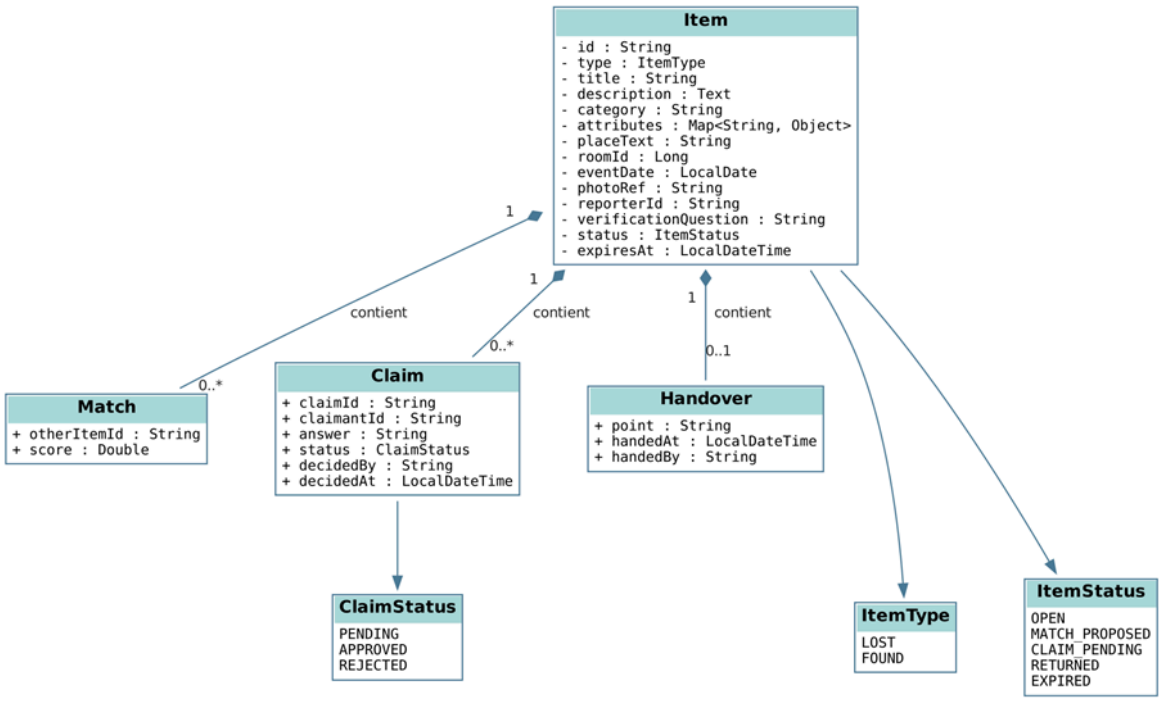


Figure 6.7 : Diagramme de classes de ms-objets-perdus

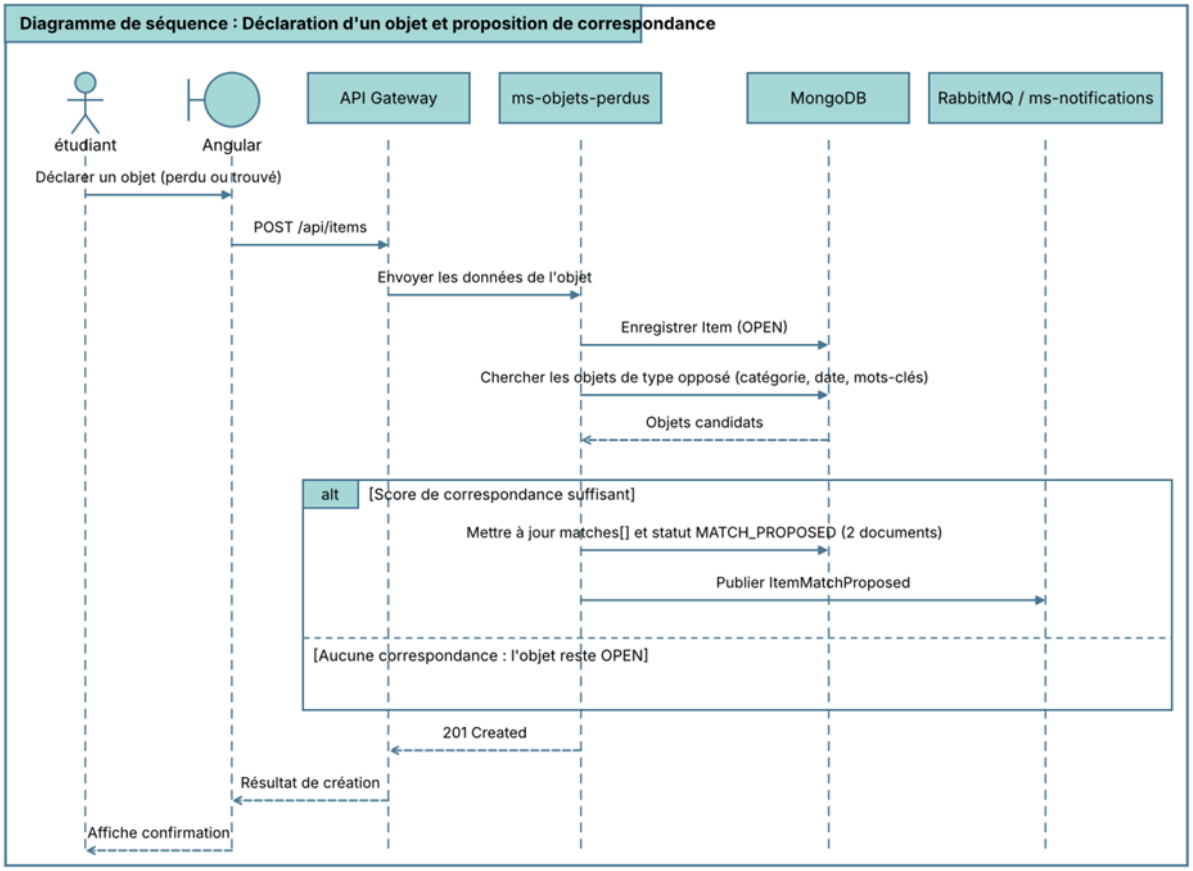


Figure 6.8 : Diagramme de séquence de ms-objets-perdus

6.5 ms-notifications

Le service reçoit les événements des autres services, détermine les destinataires, applique leurs préférences, crée les notifications, programme les rappels et tente une livraison en temps réel.

6.5.1 Modèle de données

Entité	Attributs principaux
Notification	id, recipientId, category, type, title, message, referenceType, referenceId, createdAt, deliveredAt, readAt
NotificationPreference	id, userId, category, enabled (unicité sur userId + category)
ScheduledReminder	id, recipientId, eventId, eventTitle, eventStartAt, scheduledAt, status (SCHEDULED, SENT, CANCELLED)
ProcessedMessage	messageId, eventType, processedAt

Types de notification : RESERVATION_APPROVED, RESERVATION_REJECTED, EVENT_PUBLISHED, EVENT_CANCELLED, REGISTRATION_CONFIRMED, WAITLIST_PROMOTED, CONTENT_APPROVED, CONTENT_REJECTED, ITEM_MATCH_PROPOSED, CLAIM_APPROVED, CLAIM_REJECTED, ITEM_RETURNED, EVENT_REMINDER.

Règles métier

- un même messageId n'est traité qu'une fois ; un rappel n'est envoyé qu'une fois ;
- une notification appartient à un seul destinataire, qui est le seul à pouvoir la consulter ou la marquer comme lue ;
- les préférences sont appliquées avant la création ;
- l'annulation d'une inscription ou d'un événement annule les rappels concernés.

Traitement d'un message

1. Vérifier dans ProcessedMessage que le message n'a pas déjà été traité.
2. Vérifier les préférences du destinataire.
3. Créer la Notification (et le ScheduledReminder si besoin) et enregistrer ProcessedMessage, dans une seule transaction.
4. Valider la transaction, puis envoyer l'acquittement (ACK) RabbitMQ.
5. Tenter l'envoi par SSE. Un échec SSE n'annule rien.

Rappels : un planificateur s'exécute régulièrement (par exemple chaque minute), cherche les rappels SCHEDULED dont scheduledAt est dépassé, crée une notification EVENT_REMINDER et passe le rappel à SENT dans la même transaction.

SSE : Angular ouvre une connexion sur /api/notifications/stream. Si l'utilisateur est connecté, il reçoit la notification immédiatement. Sinon elle reste en base et il la récupère par l'API REST à sa reconnexion. Le flux SSE n'est donc pas la source de vérité : c'est la base de ms-notifications.

Destinataires : certains messages contiennent directement le userId. Pour EventPublished, le service appelle ms-evenements pour récupérer les membres actifs, afin d'éviter de transporter une longue liste dans RabbitMQ.

6.5.2 API REST

Méthode	URI	Fonction	Codes
GET	`/api/notifications`	Lister ses notifications	200
GET	`/api/notifications/{id}`	Consulter une notification	200, 403, 404
PATCH	`/api/notifications/{id}/read`	Marquer comme lue	200, 403, 404
PATCH	`/api/notifications/read-all`	Tout marquer comme lu	200
GET	`/api/notification-preferences`	Consulter ses préférences	200
PUT	`/api/notification-preferences`	Modifier ses préférences	200, 400
GET	`/api/notifications/stream`	Ouvrir le flux SSE	200

6.5.3 Événements et interactions

- **Consomme :** tous les événements métier de la section 5.3 ; chaque consommateur transforme l'événement en une ou plusieurs notifications.
- **Publie :** aucun événement.
- **Appelle (synchrone) :** ms-evenements pour les membres actifs d'un club. Si ms-evenements est indisponible, le message est réessayé puis envoyé en Dead Letter Queue.

6.5.4 Diagrammes

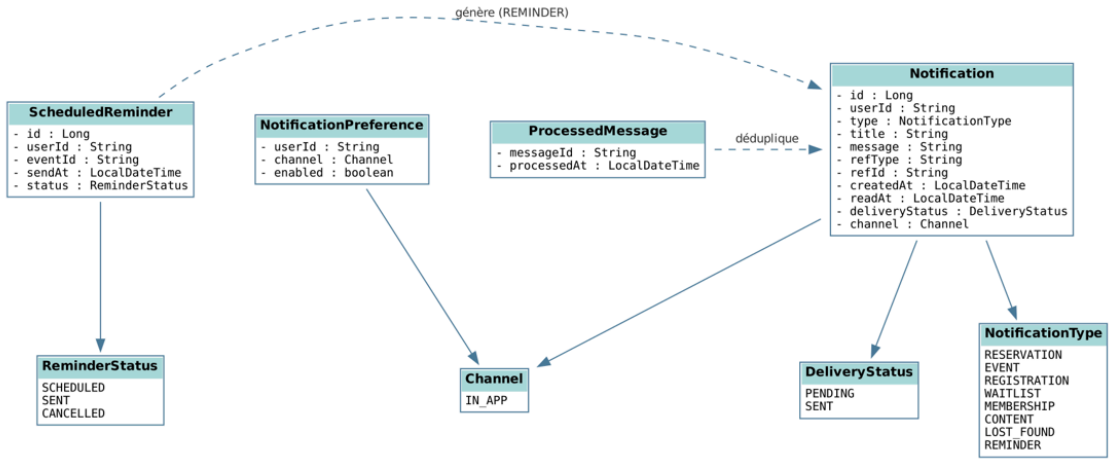


Figure 6.9 : Diagramme de classes de ms-notifications

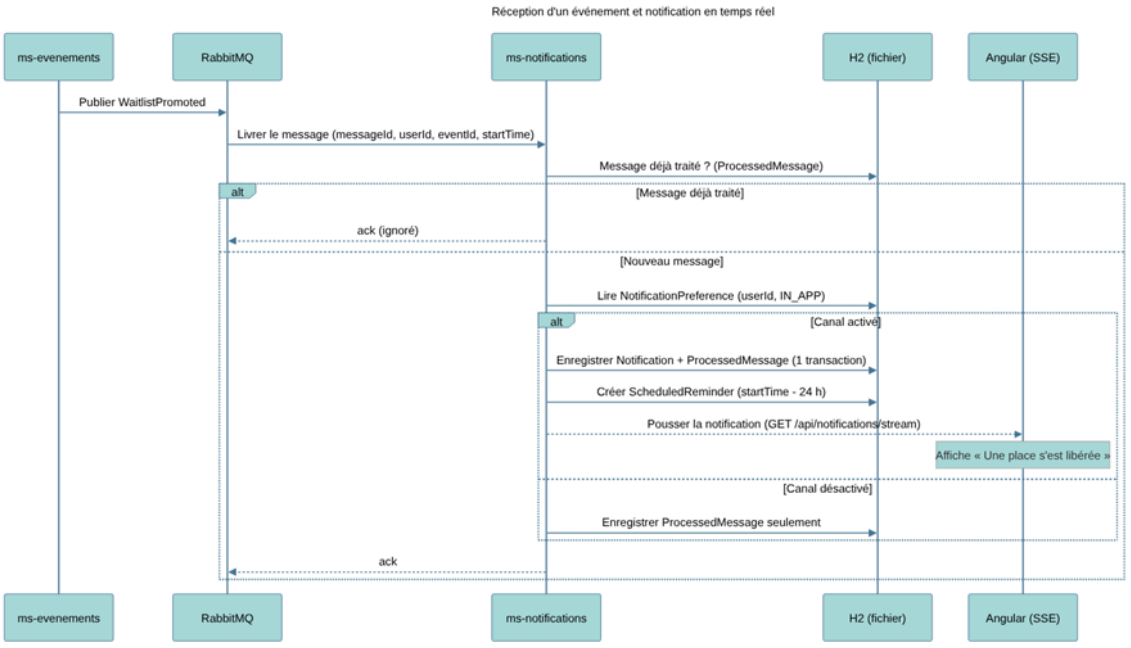


Figure 6.10 : Diagramme de séquence de ms-notifications

7. Organisation du projet

7.1 Dépôt Git

Le dépôt est public et l'enseignant y est ajouté comme collaborateur. Il respecte la structure demandée :

```
smart-campus-4das11-g2/
├── Backend/
│   ├── ms-reservations/
│   ├── ms-evenements/
│   ├── ms-annonces/
│   ├── ms-objets-perdus/
│   ├── ms-notifications/
│   ├── api-gateway/
│   ├── eureka-server/
│   └── config-server
│
├── Frontend/
├── Documentation/
│   ├── Presentation/
│   └── Diagrams/
└── README.md
```

Le README présente le sujet, les membres, l'architecture et la procédure de lancement. Le dossier Documentation contient la présentation, les diagrammes et ce dossier technique.

Méthode de travail : chaque fonctionnalité est développée dans une branche dédiée, puis intégrée par Pull Request relue par un autre membre. Chaque membre commit depuis son propre compte pour que les contributions soient traçables.

7.2 Outil de gestion de projet

Nous utilisons un tableau GitHub Projects dans lequel chaque tâche est une issue assignée à un membre. Les remarques reçues lors de la validation sont transformées en issues.

7.3 Répartition des tâches

Chaque membre est responsable d'un microservice : analyse du domaine, conception, diagramme de classes, base de données, API, interactions, initialisation du projet, commits et présentation orale de son service. Le microservice commun ms-notifications est développé par toute l'équipe et présenté à l'oral par Yassine.

Membre	Microservice	Contribution à ms-notifications
Bechir	ms-reservations (PostgreSQL)	Consommateurs des événements de réservation
Dhia	ms-evenements (MySQL)	Consommateurs des événements, inscriptions et listes d'attente
Yassine	ms-annonces (H2)	Notifications liées aux contenus ; présentation orale du service commun

Membre	Microservice	Contribution à ms-notifications
Chadha	ms-objets-perdus (MongoDB)	Notifications liées aux objets, réclamations et restitutions
Toute l'équipe	ms-notifications	Modèle commun, SSE, préférences, rappels, retry et DLQ

Les composants communs du projet, notamment l'architecture globale, l'API Gateway, Eureka Server, Config Server, Keycloak, RabbitMQ, Docker Compose ainsi que la documentation et la préparation de la présentation, sont réalisés collectivement par l'ensemble de l'équipe.

Le frontend repose sur une application Angular unique et commune, dont les différents modules fonctionnels sont répartis entre les membres selon les domaines métier dont ils sont responsables.

7.4 Planning prévisionnel

Période	Objectif	Résultat attendu
Semaine 1	Choix du sujet	Problématique et acteurs
Semaine 2	Découpage fonctionnel	Microservices et bases de données
Semaine 3	Conception	API, données et diagrammes
Semaine 4	Validation intermédiaire	Dossier, présentation et dépôt Git
Semaines 5 et 6	Développement métier	Parcours prioritaires
Semaine 7	Communications	OpenFeign et RabbitMQ
Semaine 8	Sécurité et intégration	Keycloak, Gateway et Angular
Semaine 9	Tests et stabilisation	Version démontrable
Fin du projet	Finalisation	Documentation et soutenance

Jalons

Jalon	Critère de réussite
J1 : conception validée	Responsabilités, données et API approuvées
J2 : projets initialisés	Tous les projets démarrent
J3 : infrastructure disponible	Gateway, Eureka, Config Server, Keycloak et RabbitMQ fonctionnent
J4 : parcours événement	Réservation, approbation, inscription et notification
J5 : parcours objet	Déclaration, correspondance, réclamation et restitution
J6 : version intégrée	Angular communique avec les services
J7 : version finale	Tests et documentation terminés

Figure 7.1 : Planning prévisionnel du projet

8. Conclusion

Smart Campus centralise la gestion des réservations, des clubs et événements, des annonces, des objets perdus et des notifications. La solution repose sur quatre microservices métier et un microservice commun, chacun avec sa propre base : PostgreSQL pour les réservations concurrentes, MySQL pour les clubs et inscriptions, MongoDB pour les objets aux caractéristiques variables, et H2 pour les annonces et les notifications au stade du prototype.

Les appels synchrones sont limités aux cas où une réponse immédiate est nécessaire. Les événements RabbitMQ découplent les décisions métier des notifications et des traitements secondaires. Keycloak, l'API Gateway, Eureka et Config Server complètent l'architecture.

À ce stade, l'objectif n'est pas une application terminée mais une conception cohérente, une répartition claire du travail et un dépôt Git organisé. Ce dossier sert de référence commune pour le développement, l'intégration et les tests.